CS0065 INTELLIGENT SYSTEMS

Technical Assessment 4 Knowledge Representation and Reasoning

Student: Calingasan, Dwayne Giusseppe P.
Section: AN41
Professor: Ms. Tan
Date: October 3, 2026

Classroom comfort adviser

This program compares rule-based reasoning and case-based reasoning for the same classroom situation. RBR returns every action whose condition matches. CBR retrieves a similar past situation, reuses its solution, accepts a manual revision, and retains the new case. In the demonstration, CBR needs an added cooling action to match the classroom needs identified by RBR.

Part 1 Knowledge base and use case

The Python dictionary stores six facts in three categories: **climate** contains temperature and humidity; **weather** contains rain and daylight status; **room** contains occupancy and window status. Values describe a hot, humid, occupied classroom with rain outside, no daylight, and open windows. These are illustrative teaching data, not sensor readings.

The current facts form the working memory. Three rule functions form the RBR rule base. The inference loop evaluates every rule and collects all inferred actions. Humidity contributes to case similarity, although the three RBR rules do not use it.

How to run

`python TA4_Calingasan.py`
Enter a replacement solution when prompted, or press Enter to keep it.
`python TA4_Calingasan.py --demo`
Reproduces the report output using a scripted revision. Python 3 and its standard library are sufficient. Edit FACTS to test other classroom situations.

Python source 1 of 4 Knowledge and cases

```
"""TA4: Knowledge representation, RBR, and CBR.

Run: python TA4_Calingasan.py
Reproduce the report example: python TA4_Calingasan.py --demo
The classroom facts and sample cases are illustrative teaching data.
"""

import argparse
from copy import deepcopy


# PART 1: Six facts grouped into three explicit categories.
FACTS = {
    "climate": {"temperature": "hot", "humidity": "high"},
    "weather": {"is_raining": True, "is_daylight": False},
    "room": {"is_occupied": True, "windows_open": True},
}

# Three illustrative past situations, each with features and a solution.
CASE_BASE = [
    {
        "id": "C1",
        "problem": {
            "climate": {"temperature": "hot", "humidity": "high"},
            "weather": {"is_raining": False, "is_daylight": True},
            "room": {"is_occupied": True, "windows_open": False},
        },
        "solution": "Turn on the aircon.",
    },
    {
        "id": "C2",
        "problem": {
            "climate": {"temperature": "warm", "humidity": "high"},
            "weather": {"is_raining": True, "is_daylight": False},
            "room": {"is_occupied": True, "windows_open": True},
        },
        "solution": "Close the windows. Turn on the lights.",
    },
    {
        "id": "C3",
        "problem": {
            "climate": {"temperature": "cool", "humidity": "normal"},
            "weather": {"is_raining": False, "is_daylight": True},
            "room": {"is_occupied": False, "windows_open": False},
        },
        "solution": "Keep the aircon and lights off.",
    },
]
```

Python source 2 of 4 Rule based reasoning

```
# PART 2: Each function is one explicit IF-THEN rule.
def rule_hot(facts):
    if facts["climate"]["temperature"] == "hot" and facts["room"]["is_occupied"]:
        return "Turn on the aircon."


def rule_rain(facts):
    if facts["weather"]["is_raining"] and facts["room"]["windows_open"]:
        return "Close the windows."


def rule_dark(facts):
    if not facts["weather"]["is_daylight"] and facts["room"]["is_occupied"]:
        return "Turn on the lights."


RULES = (rule_hot, rule_rain, rule_dark)


def infer(facts):
    """Check EVERY rule against working memory; return all actions."""
    actions = []
    for rule in RULES:
        action = rule(facts)
        print(f"  {rule.__name__}: {'MATCH' if action else 'NO MATCH'}")
        if action is not None:
            actions.append(action)
    return actions
```

Python source 3 of 4 Case based reasoning

```
# PART 3: Retrieve, reuse, revise, and retain.
def flatten(facts):
    """Use category/feature pairs to preserve the knowledge structure."""
    return {(category, key): value
            for category, values in facts.items()
            for key, value in values.items()}


def similarity(problem, past_problem):
    """Equal-weight exact matches / number of features in the query."""
    query, past = flatten(problem), flatten(past_problem)
    if not query:
        raise ValueError("The new problem must contain at least one fact.")
    matches = sum(key in past and value == past[key]
                  for key, value in query.items())
    return matches / len(query)


def retrieve(problem, case_base):
    """Return the best case; the earliest case wins a similarity tie."""
    if not case_base:
        raise ValueError("The case base must not be empty.")
    scored = [(case, similarity(problem, case["problem"]))
              for case in case_base]
    for case, score in scored:
        print(f"  {case['id']}: similarity = {score:.2%}")
    return max(scored, key=lambda item: item[1])


def reuse(case):
    """Start with the retrieved case's existing solution."""
    return case["solution"]


def revise(solution, replacement):
    """Keep the solution when the user submits an empty response."""
    return replacement.strip() or solution


def retain(case_base, problem, solution):
    """Store an independent copy for reuse during this program session."""
    case = {"id": f"C{len(case_base) + 1}",
            "problem": deepcopy(problem), "solution": solution}
    case_base.append(case)
    return case
```

Python source 4 of 4 Program execution

```
def main(demo=False):
    # Working memory contains the current classroom observations.
    facts, case_base = deepcopy(FACTS), deepcopy(CASE_BASE)
    print("TA4 | CLASSROOM COMFORT ADVISER")
    print("INPUT: current classroom facts")
    for category, values in facts.items():
        print(f"  {category}: {values}")

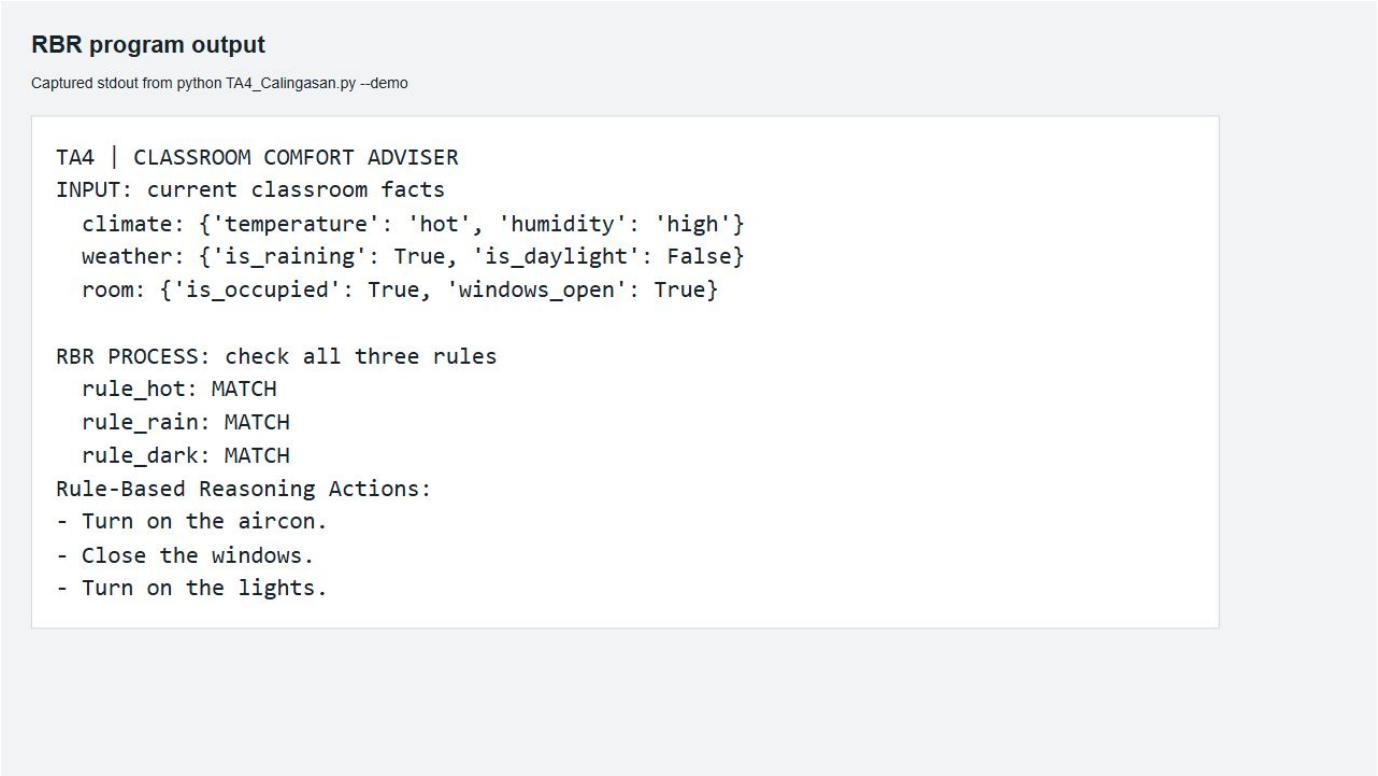
    print("\nRBR PROCESS: check all three rules")
    actions = infer(facts)
    print("Rule-Based Reasoning Actions:")
    for action in actions:
        print(f"- {action}")
    if not actions:
        print("- No actions inferred.")

    print("\nCBR PROCESS: RETRIEVE")
    case, score = retrieve(facts, case_base)
    print(f"Selected case: {case['id']} ({score:.2%})")
    solution = reuse(case)
    print(f"REUSE: {solution}")
    if demo:
        # Scripted manual revision to make the report reproducible.
        replacement = solution + " Turn on the aircon."
        print(f"REVISE (demo input): {replacement}")
    else:
        replacement = input("REVISE: new solution, or Enter to keep: ")
    solution = revise(solution, replacement)
    print(f"Final CBR solution: {solution}")
    new_case = retain(case_base, facts, solution)
    print(f"RETAIN: added {new_case['id']}; case count = {len(case_base)}")
    print("Retention is in memory for this run; no file is written.")
    print("\nVerification: retrieve the same problem after retention")
    best, score = retrieve(facts, case_base)
    print(f"Selected case: {best['id']} ({score:.2%})")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--demo", action="store_true",
                        help="Use the report's scripted revision.")
    main(parser.parse_args().demo)
```

Part 2 Rule based reasoning output

The input facts trigger all three rules: an occupied hot room needs cooling; rain with open windows requires closing them; an occupied room without daylight needs lighting. The engine checks every rule rather than stopping after the first match.



Screenshot of the captured program stdout displayed in a local browser. The output was produced by the submitted Python file in demonstration mode.

Part 3 Case based reasoning output

Similarity is the number of exactly matching query features divided by six, with equal weight per feature. C1 matches 3/6 features, C2 matches 5/6, and C3 matches 0/6. C2 is retrieved, but its warm-room solution omits cooling. The demonstration revision adds the aircon action.

CBR program output

Captured stdout from python TA4_Calingasan.py --demo

```
CBR PROCESS: RETRIEVE
  C1: similarity = 50.00%
  C2: similarity = 83.33%
  C3: similarity = 0.00%
Selected case: C2 (83.33%)
REUSE: Close the windows. Turn on the lights.
REVISE (demo input): Close the windows. Turn on the lights. Turn on the aircon.
Final CBR solution: Close the windows. Turn on the lights. Turn on the aircon.
RETAIN: added C4; case count = 4
Retention is in memory for this run; no file is written.

Verification: retrieve the same problem after retention
  C1: similarity = 50.00%
  C2: similarity = 83.33%
  C3: similarity = 0.00%
  C4: similarity = 100.00%
Selected case: C4 (100.00%)
```

The retained case C4 matches all six features on a second retrieval. Retention lasts for the current run; restarting restores the three initial cases. The screenshot shows the captured stdout, including the scripted revision input.

Part 4 Comparative reflection

RBR was easier to implement in this program because each condition translated directly into an IF statement, and one loop collected the matching actions. CBR needed additional code for similarity scoring, case selection, revision, and retention. CBR is more flexible for varied or less structured problems because it can adapt a similar experience rather than require an exact rule; however, this implementation still requires six structured features and human judgment during revision. For continuously changing patterns, CBR is often easier to extend by retaining useful new cases, while outdated or poor cases must be reviewed. If only the input readings change and the decision policy stays fixed, RBR remains a good choice because the same rules can be evaluated against each new observation. RBR applications include appliance control and rule-driven troubleshooting. CBR applications include help-desk support that reuses resolved tickets and equipment diagnosis based on earlier repair cases.

Observed differences

RBR immediately inferred cooling, window closure, and lighting. CBR initially returned window closure and lighting because the closest historical case described a warm room. Adding cooling in the revise step produced the same action set as RBR. Retaining that solution made the next retrieval an exact match. The 83.33% score measures feature agreement; it is not a probability that a solution is correct.

Limitations and improvements

All six features currently have equal weight, so a less important match can count as much as occupancy or temperature. Ties select the earliest stored case. A production version could weight important features, require a minimum similarity, review case quality, and save retained cases to a file or database. Manual revisions are accepted as text without checking whether the proposed actions are correct; reviewing them before retention would improve reliability.

Verification

The program passed checks for all three rules individually, all rules matching, no rules matching, known similarity scores, tie handling, retrieval, reuse, unchanged and edited revisions, retention, independent copies of stored facts, and empty-input errors. Both interactive revision paths were also run through the actual command-line program. The demonstration output was captured from a successful program run.